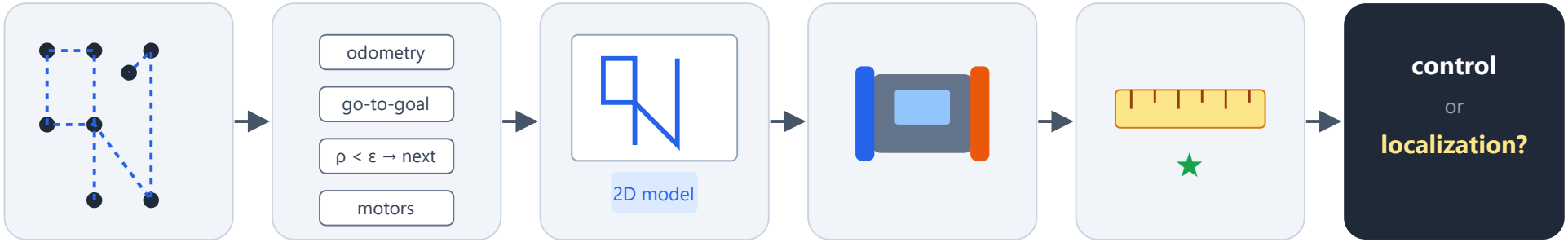
Go-to-goal

The robot drives my student-ID digits **9 1** as a list of waypoints, steering by its own odometry. We compare the plan, the simulator, the real robot's log, and a tape measurement

The whole thing in one picture



1 · 8 waypoints
(x, y) list for "91"

2 · Block program
one loop

3 · Simulator
tune the gains

4 · Real robot
same program

5 · Tape measure
true end point

6 · Colab
Lab7_go_to_goal.ipynb

Question
Given only "go to (x, y)", can the robot draw a shape? And when it misses, is the **controller** to blame or the **pose estimate**?

Method
Go-to-goal controller on top of the Lab 6 odometry. Tune it in the simulator, run the same program and gains on the robot, then measure the end point with a tape.

Answer in short
The controller works: it hit every waypoint of its own estimate within 1 cm. The estimate itself was 29 cm off: a **localization** problem.

Result at a glance

worst waypoint miss (robot's own log)
2.7 / 0.8 cm
simulator / real robot

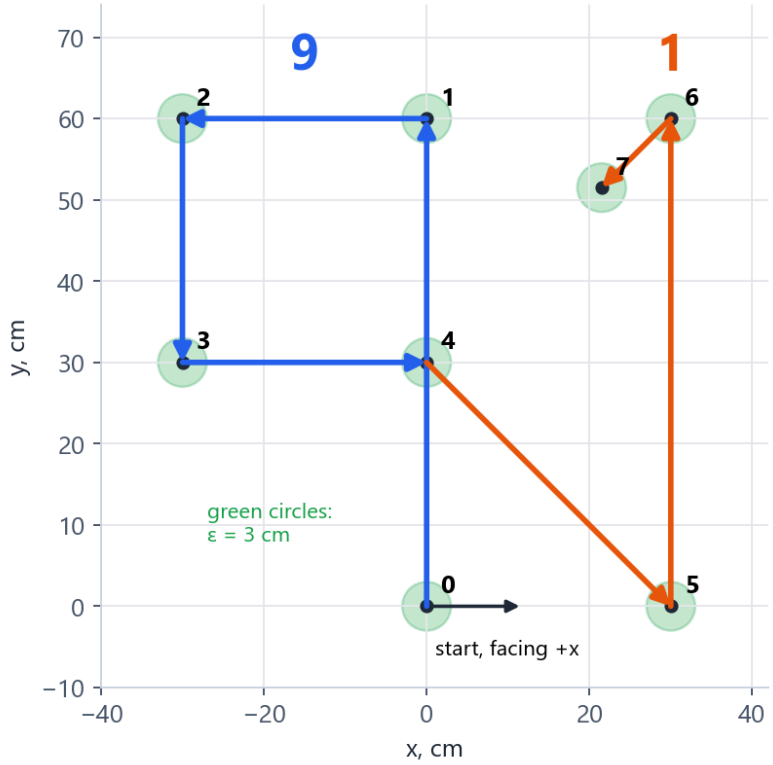
real robot: odometry vs tape
29 cm · 48°
after 2.9 m of driving

run time
30.9 / 17.4 s
simulator / real robot

loop period
20 / 57 ms
simulator / real robot

1 The digits as waypoints

a shape is just a list of points



#	X, M	Y, M	WHAT
0	0	0	start (reached at once)
1	0	0.60	9: up the stem
2	−0.30	0.60	9: top of the loop
3	−0.30	0.30	9: down
4	0	0.30	9: close the loop
5	0.30	0	gap: diagonal to the foot of the 1
6	0.30	0.60	1: up
7	0.215	0.515	1: flag (12 cm at 45°)

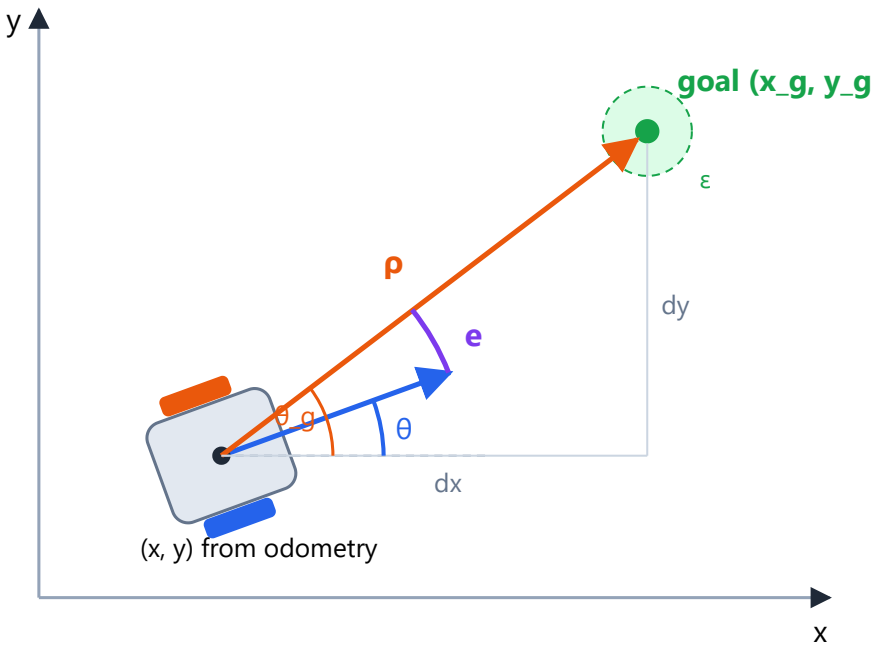
```
-- init block (arrays index from 0)
wx = {0, 0, -0.30, -0.30, 0, 0.30, 0.30, 0.215}
wy = {0, 0.60, 0.60, 0.30, 0.30, 0, 0.60, 0.515}
N = 8
```

Frame = the robot's start pose: **x** = the direction it faces at the start, **y** = to its left. Same as the Lab 6 odometry, so the waypoints can be compared with the log directly.

Lab 6 drove the same shape with a **segment** controller ("straight 0.6 m, turn 90°…"). Here the robot only knows **where to go next**; the controller works out the rest.

2 The go-to-goal controller

distance and bearing to the goal, every loop pass



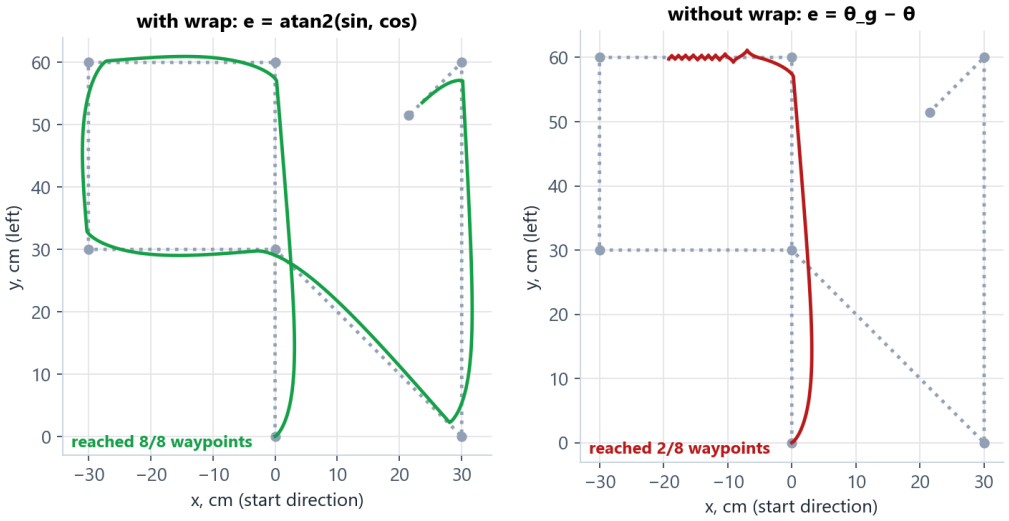
QUANTITY	FORMULA
distance to the goal	$\rho = \sqrt{(dx^2 + dy^2)}$
bearing to the goal	$\theta_g = \mathrm{atan2}(dy, dx)$
heading error	$e = \mathrm{wrap}(\theta_g - \theta)$
forward	$v = \min(K_v \cdot \rho, v_max)$
turning	$\omega = K_\theta \cdot e$
wheels	$v_R = v + \omega \cdot B/2,\ v_L = v - \omega \cdot B/2$

e is the angle the robot still has to turn: goal direction θ_g minus its own heading θ. If e > 0 the goal is to the left, so the right wheel speeds up.

ρ sets the speed: far away → fast, close → slow, so the robot does not race past the waypoint.

When **ρ < ε** (green circle) the robot counts the waypoint as reached and aims at the next one. Nothing else is planned: the path between waypoints comes out of the control law.

3 Wrapping the heading error



Same controller, same waypoints, wrap removed. The heading θ keeps growing past 180° (it is the integral of the turns, 592° by the end of this path), so θ_g − θ can be off by ±360°. Going left at waypoint 2, θ_g jumps between +180° and −180°: the robot sees e ≈ 0, then e ≈ −360°, and chatters on the spot. It never gets past waypoint 2.

Example

Heading θ = 170°, goal at θ_g = −170°. Raw error = −340° → "turn 340° right". The goal is really only **20° to the left**.

$$\text{wrap}(-340°) = +20°$$

Three ways to wrap into (−π, π]

```
-- 1. while loops
while e > pi:   e = e - 2*pi
while e <= -pi: e = e + 2*pi

-- 2. atan2 (used in my program: no if/while needed)
e = atan2(sin(e), cos(e))

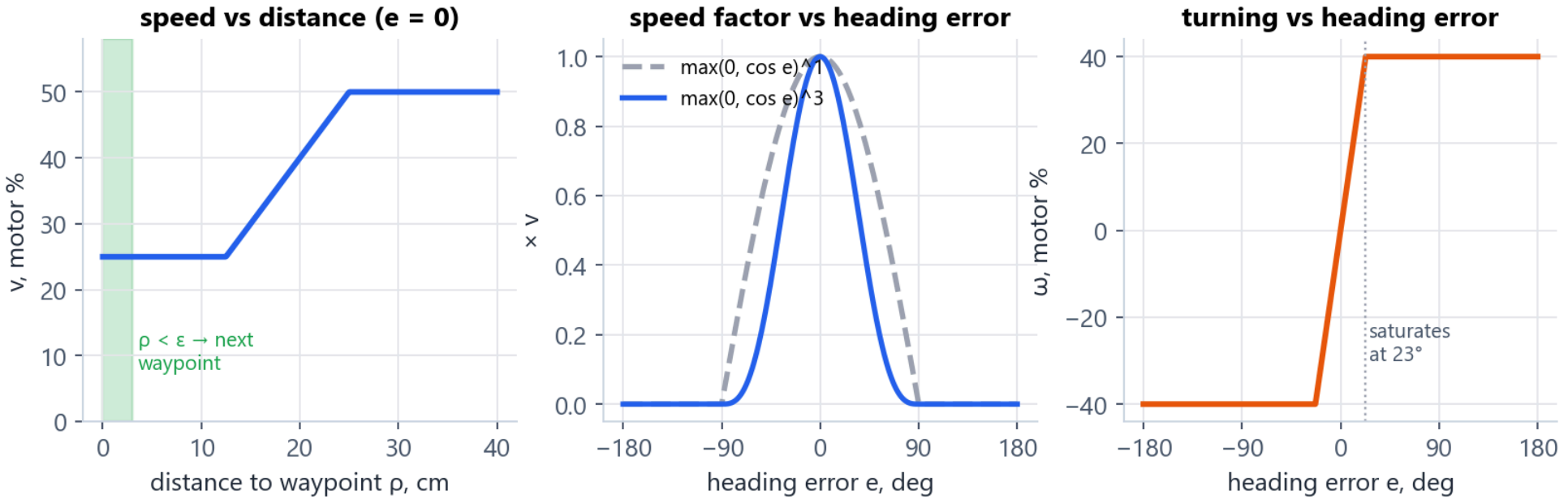
-- 3. modulo (Python: % is always ≥ 0 for 2π)
e = (e + pi) % (2*pi) - pi
```

Why atan2 works: sin and cos do not change when you add 360°, and atan2 returns the one angle in (−π, π] with that sine and cosine. Function blocks have no `if`/`while`, so this is the natural choice in TRIK.

The professor's two examples: the .py uses method 1 (`while`); `Fixed_1x1_trajectory.qrs` uses one step of it, `err − sgn(err)·2π` inside an If block, which is enough only when |err| < 3π.

4 From the formula to motor power

the shapes of v and ω




```
-- step block, after the odometry
e = atan2(sin(theta_g - course), cos(theta_g - course))
v = active * max(VMIN, min(KV * rho, VMAX)) * max(0, cos(e)) ^ CP
w = active * max(-TMAX, min(KT * e, TMAX))
pL = v - w          -- M4, left
pR = v + w          -- M3, right
```

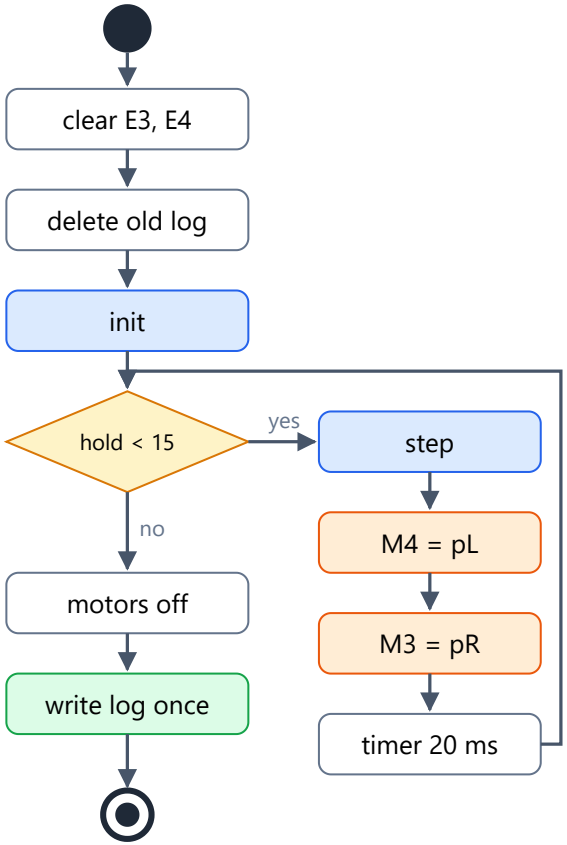
The motors take **power in %**, not m/s, so ω·B/2 is folded into K_θ: pR = v + ω, pL = v − ω.

GAIN	VALUE	WHY
K_v	200 %/m	full 50 % until 25 cm away, then slow down
VMIN	25 %	floor dead zone is 12–15 % (Lab 5): below that the robot stalls near a waypoint
K_θ	100 %/rad	turn hard; saturates at ±40 % for |e| > 23°
cos(e)³	—	goal behind or to the side → v = 0: turn on the spot first, then drive
ε	3 cm	switch late enough that corners stay sharp

Turn-then-drive vs drive-while-turning (lecture): cos(e)³ is a smooth mix. At e = 45° the speed is already down to 35 %, at 90° it is 0. With plain cos(e) it would still be 71 % at 45°, and the corners turn into arcs.

5 The block program

go_to_goal_sim.qrs = go_to_goal_real.qrs, except r, B and the log name



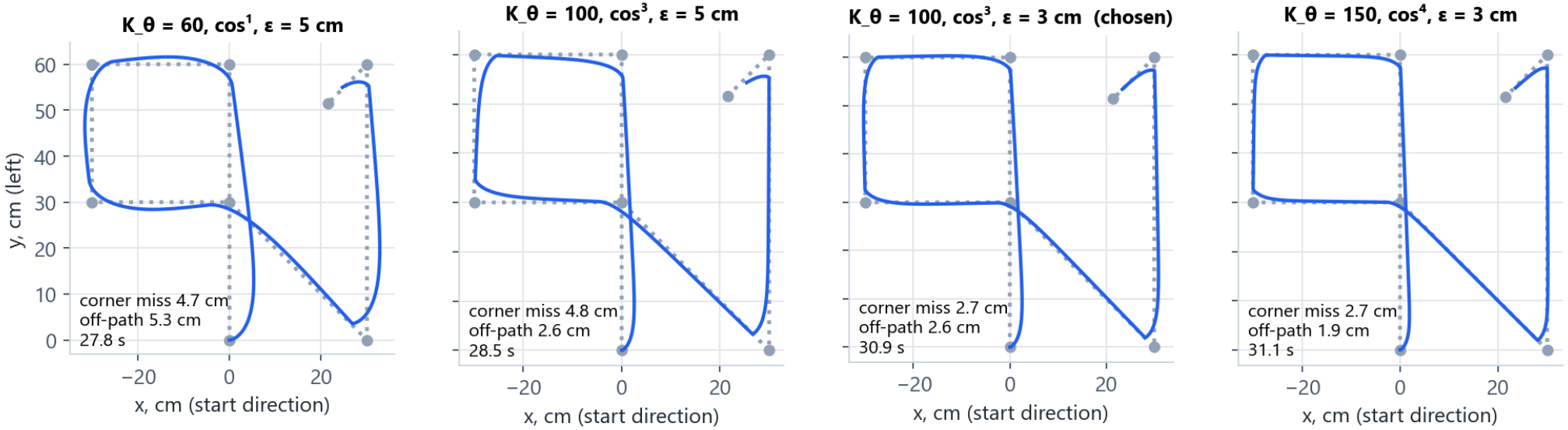

```
-- odometry (Lab 6): wheel angles in rad, left = E4, right = E3
theta_l = encoder4 * pi / 180
theta_r = encoder3 * pi / 180
prev_path = path
path = (theta_r + theta_l) * r / 2
course = (theta_r - theta_l) * r / B
x = x + (path - prev_path) * cos(course)
y = y + (path - prev_path) * sin(course)
-- go-to-goal towards waypoint k
active = max(0, sgn(N - i))      -- 1 until the last waypoint
k = min(i, N - 1)
dx = wx[k] - x
dy = wy[k] - y
rho = sqrt(dx * dx + dy * dy)
theta_g = atan2(dy, dx)
e = atan2(sin(theta_g - course), cos(theta_g - course))
v = active * max(VMIN, min(KV * rho, VMAX)) * max(0, cos(e)) ^ CP
w = active * max(-TMAX, min(KT * e, TMAX))
pL = v - w
pR = v + w
-- waypoint reached?  (no if in Function blocks)
i = i + active * max(0, sgn(EPS - rho))
hold = hold + (1 - active)
buf = buf .. x * 100 .. "," .. y * 100 .. "," .. t .. "\n"
```

No `if`: `max(0, sgn(EPS − rho))` is 1 inside the circle and 0 outside, so `i` goes up by one exactly when the waypoint is reached.

Log: `x, y` in cm and time in ms, kept in a string and written once after the loop (README format, no header).

6 Tuning in the simulator

the lecture's "design choices", measured



K_Θ	SPEED FACTOR	E	TIME	WORST CORNER MISS	WORST OFF-PATH
60	cos e	5 cm	27.8 s	4.7 cm	5.3 cm
100	$\cos^3$ e	5 cm	28.5 s	4.8 cm	2.6 cm
100	**$\cos^3$ e**	**3 cm**	**30.9 s**	**2.7 cm**	**2.6 cm**
150	$\cos^4$ e	3 cm	31.1 s	2.7 cm	1.9 cm

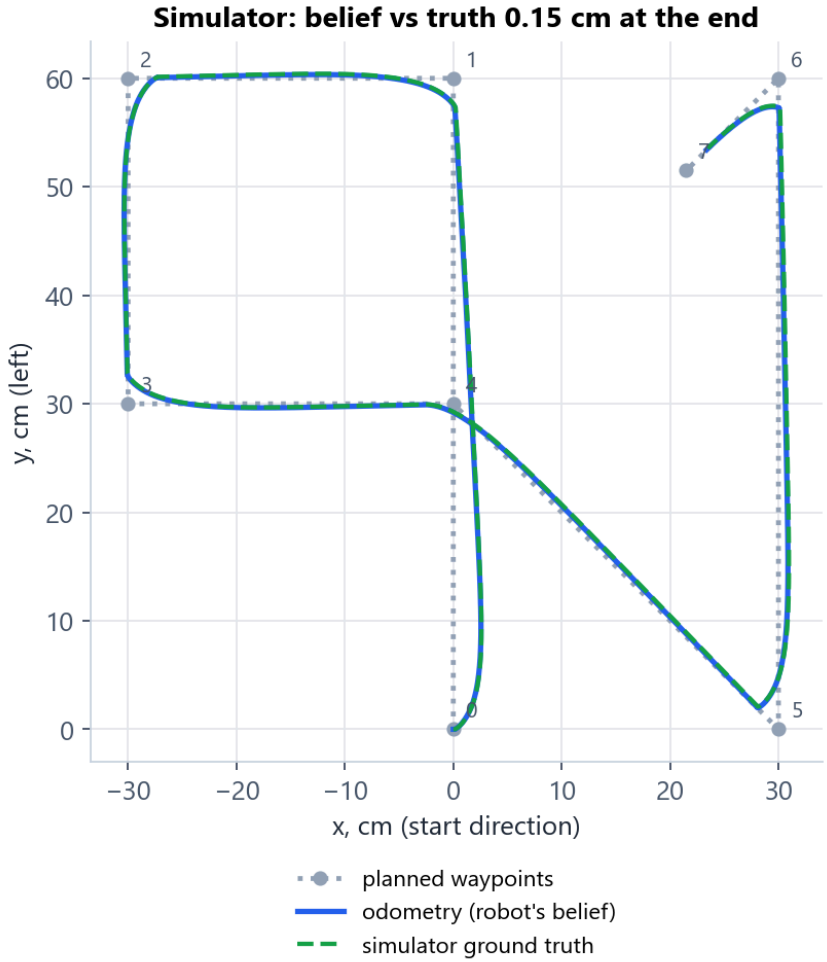
K_θ low + plain cos: wide arcs after each corner, the lines bow by 5 cm ("K_θ too low").

Large ε: the waypoint is "reached" 5 cm early, so every corner is cut by about ε ("large ε rounds corners").

Chosen: sharpest corners for +3 s. K_θ = 150 is a bit tighter, but on the real robot (57 ms loop) a higher gain is closer to oscillating.

7 Simulator result

belief = truth, so this is pure control



WAYPOINT	X, Y (CM)	CLOSEST PASS, SIM
0	0, 0	0.0 cm
1	0, 60	1.8 cm
2	−30, 60	1.8 cm
3	−30, 30	1.8 cm
4	0, 30	0.7 cm
5	30, 0	2.7 cm
6	30, 60	2.7 cm
7	21.5, 51.5	2.7 cm

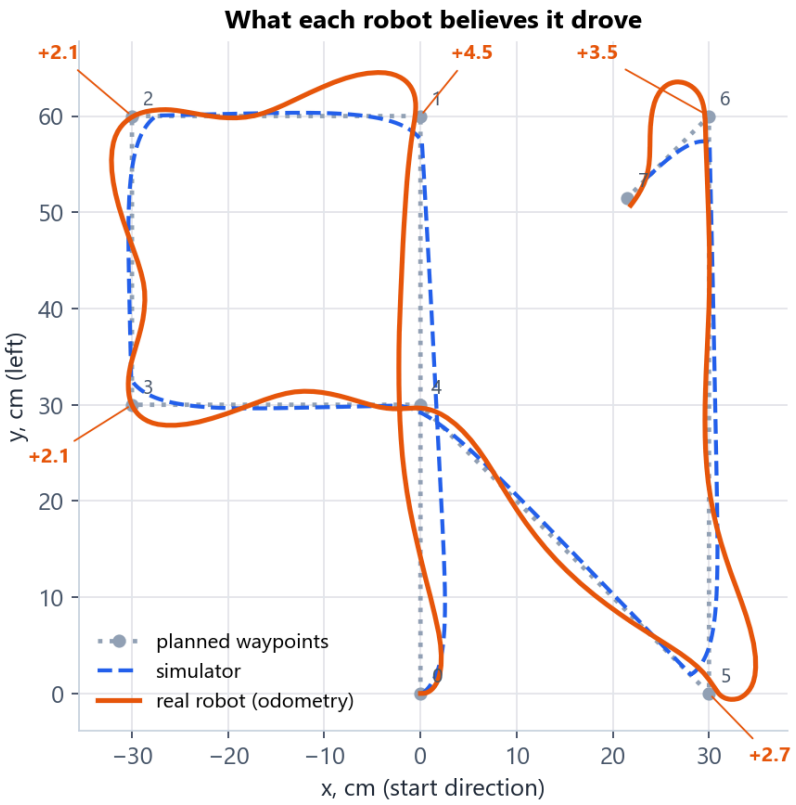
All misses are **below ε = 3 cm**: the robot drives at the waypoint, enters the 3 cm circle, and turns to the next one from there. That is by design, not an error.

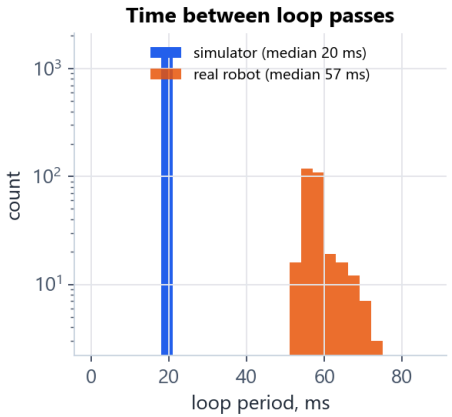
The odometry and the simulator's ground truth agree to **0.15 cm** at the end (track B = 15.4 cm, found in Lab 6). In the simulator there is no localization error, so whatever is left is the controller.

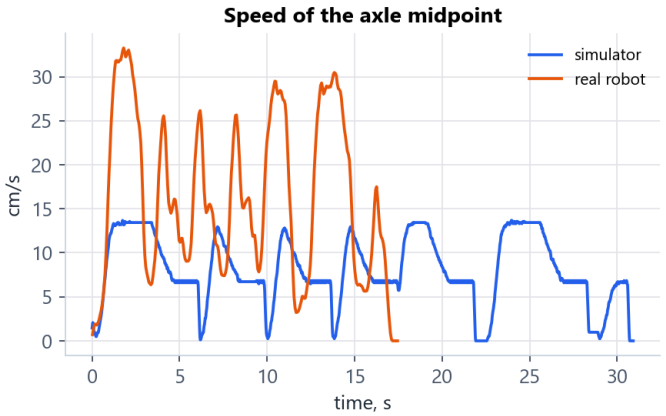
The bow on the first stem (2.6 cm to the right): the robot starts facing +x and the first goal is 90° to its left, so it turns on the spot, but it is still turning slightly when it starts to move.

8 Real robot: what it believes

trik-6fc169, 2026-10-29, battery 12.3 V, same program and gains







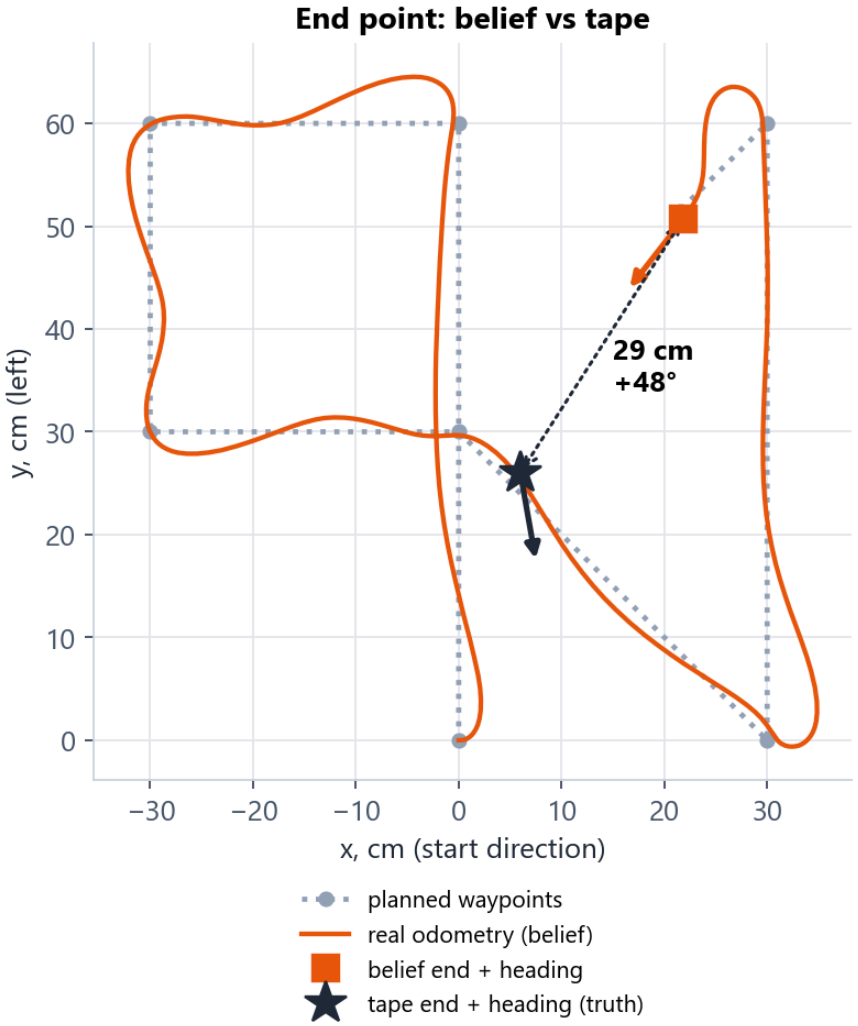
	SIM	REAL (ODOMETRY)
worst waypoint miss	2.7 cm	0.8 cm
corner overshoot (1, 2, 3, 5, 6)	≤ 0.4 cm	2.1–4.5 cm
worst distance from the lines	2.6 cm	5.4 cm
run time · path length	30.9 s · 2.5 m	17.4 s · 2.9 m

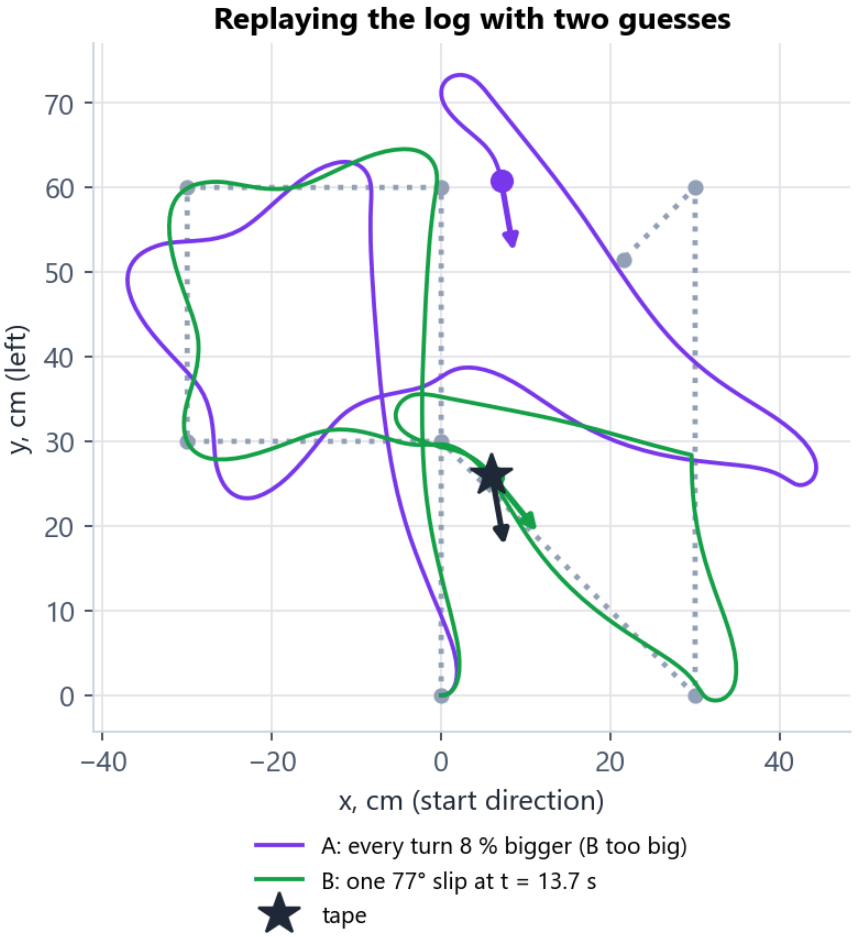
Why it overshoots and weaves: the controller only updates every **57 ms** (20 ms in the simulator), and the motors need ~0.16 s to change speed (Lab 5). The real robot is also about twice as fast. By the time it sees "you are at the corner", it is already 2–4 cm past it; on the straights the late corrections make it weave.

Not the log: this time the log is written once at the end, and the loop is still 57 ms (Lab 6: 59 ms with a write every pass). The slow loop is the robot's diagram runtime.

9 Real robot: where it really was

tape measurement of the end pose





Belief vs truth: odometry says (21.9, 50.7) cm facing −128°; the tape says (6, 26) cm facing about −80°. **29 cm and 48° off**, after 2.9 m. The robot cannot notice.

Guess A, wrong wheel base B: 48° extra over 592° of turning = **8 %**. Every turn 8 % bigger matches the heading (B ≈ 0.18 m instead of 0.194), but the replayed path ends near (7, 61), not (6, 26).

Guess B, one slip: the best single heading jump (77° at t = 13.7 s, halfway up the 1) lands exactly on (6, 26) but ends facing −51°, not −80°. Neither simple cause fits both numbers: one run with a tape cannot separate them.

10 The two questions

from the Lab 7 deliverables

1 · Which corners were cut or overshot, and which gain or ε caused it?

- **Cut:** none with the final gains. With **ε = 5 cm** every corner was cut by up to 4.7 cm (the waypoint counts as reached too early). With **K_θ = 60** and plain cos e the robot drove wide arcs and the lines bowed by 5 cm.
- **Overshot:** on the real robot, every corner by 2–4.5 cm, worst at corner 1 (end of the longest, fastest leg). Cause: **K_θ = 100** with a **57 ms** loop and 0.16 s motor lag; the simulator, at 20 ms, overshot ≤ 0.4 cm.
- **End point:** 0.8–2.7 cm short of the flag in the logs, because the robot stops as soon as it is inside ε.

2 · Is the real-vs-model gap control or localization?

Localization. In its own odometry the real robot reached every waypoint within 0.8 cm, better than the simulator. But the tape put it 29 cm and 48° away from where it thought it was. The controller faithfully followed a wrong pose estimate: the odometry counted about 8 % too little turning (nominal B, slip in the fast spins).

ERROR	SIZE	TYPE
miss in the robot's own log	≤ 0.8 cm	control
corner overshoot / weave	2–5 cm	control (timing)
odometry vs tape, position	29 cm	localization
odometry vs tape, heading	48°	localization

> **The robot's log looks almost perfect. Only the tape shows the error: you cannot judge localization from the robot's own log.**

Compared with Lab 6 (12 cm after 3.2 m), this run drifted further: the go-to-goal robot spins on the spot at 40 % power at every corner, which is exactly where wheels slip.

11 Summary

Five takeaways

1 · Go-to-goal needs only ρ, θ_g and the heading error e. The shape comes from the waypoint list.

2 · Always wrap e. Without it the robot stuck at waypoint 2 of 8; `atan2(sin e, cos e)` fixes it in one line.

3 · ε and K_θ shape the corners: big ε cuts them, low K_θ rounds them, `cos(e)³` turns first and drives second.

4 · The real robot overshoots by 2–4.5 cm because its loop is 57 ms, not 20 ms.

5 · The controller followed its estimate within 0.8 cm, but the estimate was 29 cm / 48° off: a localization problem.

What I would do next

Calibrate B: spin 10 turns in place and compare with the encoders; the 48° says B ≈ 0.18 m, not 0.194.

Less slip: lower TMAX (40 → 25 %) so the spins are gentler, and ramp the speed up after each turn.

Better heading: a gyroscope (Week 10) measures the turn directly, independent of wheel slip.

Files: go_to_goal_sim.qrs, go_to_goal_real.qrs, odometry_data_sim.txt, odometry_data_real.txt, Lab7_go_to_goal.ipynb, README.md in `07_Go-to-goal/Lab7/`.